CNN Image Classification on CIFAR-10: How the ConvNet Architecture Changes Performance

Task 3 report · course notebook *colab_img_clf.ipynb* · 21 October 2026

Summary. The notebook as given scores 9.9–11.3% after one epoch, which is chance for ten classes, so the first part of this work was finding out why rather than changing anything. Two faults turned out to be involved. The sigmoid output trained with BCE stalls optimisation, and the data loader copies the red channel into all three input channels. Replacing only the output takes the same layers to 60.9%, and the corrected loader is worth a further 7.8 points. That fixed network, at 61.2 ± 0.4% over three seeds, is the reference I changed one element at a time against. The clearest finding is that **batch normalisation** (+5.8 points for 224 extra parameters) buys more than any amount of extra width, and that it is what lets depth pay off at all: with it, a four-block network reaches 73.5 ± 1.8% and two convolutions per block 75.1%, both beating a wider model that costs more to run (68.9 ± 1.6%). Max-pooling beat the alternatives, and dropout hurt, because after ten epochs these networks are still under-fitting. Adding augmentation and a cosine schedule to the four-block network gives **77.6%**.

1. Task and experimental protocol

The notebook trains a small network, *ConvNet*, on CIFAR-10 [1]: 32×32 colour images in ten classes. ConvNet has three blocks of 3×3 convolution, ReLU and 2×2 max-pooling with 16, 32 and 64 filters and no padding, then global average pooling (GAP) and fully connected (FC) layers 128–64–10. Its output is a sigmoid trained with binary cross-entropy (BCE) on one-hot labels, and it has 40,810 parameters. Every model was trained under the same protocol, so the architecture is the only thing that changes:

- **Data.** The notebook's split: data_batch 1, 3 and 5 for training (30,000 images), 2 and 4 for validation (20,000) and test_batch for testing (10,000). Pixels are divided by 255.
- **Training.** SGD with learning rate 0.0025, momentum 0.9 and batch size 10 exactly as in the notebook, but 10 epochs instead of 1 (Section 2 shows why one is not enough). Following the lecture's early-stopping idea I keep the lowest-validation-loss checkpoint, so model selection never sees the test set. All runs use seed 0; four key models were repeated with seeds 1 and 2 (Section 5).
- **Evaluation.** Test accuracy, macro-F1, per-class accuracy and the confusion matrix; cost as parameters, multiply–accumulate operations per image (MACs) and seconds per epoch on one idle CPU thread. *Map / RF*: last feature map / receptive field of its units, in pixels.
- **Environment.** PyTorch 2.14 on CPU, on the original CIFAR-10 batch files supplied with the course. For reference on the same test set, the course's pretrained EfficientNet checkpoint (*modelz.pth*) scores 76.2% through the notebook's own pipeline. The accompanying notebook reproduces every run.

2. Running the notebook as given

Run unchanged, ConvNet reached 9.9–11.3% in 5 seeds out of 5. My first assumption was that one epoch is simply too short, but the loss curve says something worse: the network is not learning slowly, it has stopped.

Training loss drops to about 0.325 inside the first epoch and then sits there until epoch 7, with validation accuracy flat at chance (Fig. 1a). That number is not arbitrary. It is exactly the BCE of answering p = 0.1 to all ten classes, so the network has found the class prior and has no gradient pushing it anywhere else. Averaging the BCE over ten outputs is what makes the plateau so hard to leave: each logit receives roughly a tenth of the gradient it would get under softmax cross-entropy (CE). Ten epochs of this reach 20.9%, with five classes never predicted at all. Changing nothing but the output to softmax + CE gives 60.9%. BatchNorm alone also pulls the sigmoid model out (63.0%), so the stall is partly about signal strength through six unnormalised layers, not the loss alone. Either way a model at chance cannot rank architectures, so the softmax version is the **reference** from here on.

The second fault is quieter and I only noticed it when a sample image looked wrong. In `myData.__getitem__`, `img_red`, `img_green` and `img_blue` are all sliced from `[0:1024]`, so the network receives three copies of the red channel and never sees colour. With the correct slices (`[1024:2048]` and `[2048:3072]`) the reference goes from 53.1% to 60.9%. Two-thirds of the input was being thrown away for 7.8 points. Every run below uses the corrected loader.

3. Architecture experiments

Table 1 lists every run. Each variant changes one element of the reference — capacity, receptive field, downsampling, normalisation or the head; Δ is the change in test accuracy.

Model	Change vs. reference	Map / RF	Params	MACs	s/ep	Best ep	Train %	Val %	Test %	F1 %	Δ test
Notebook model as given, and the data check											
Original (sigmoid)	as given: sigmoid + BCE output	2×2 / 22	40.8k	1.5M	15	10	20.5	20.5	20.9	12.3	−40.0
Original + BN	as given + BatchNorm after each conv	2×2 / 22	41.0k	1.5M	21	10	61.5	62.1	63.0	62.6	+2.1
Reference, red only	loader bug: red copied into G and B	2×2 / 22	40.8k	1.5M	14	10	57.3	52.5	53.1	53.0	−7.8
Reference (original layers, softmax output) and single changes, grouped by what they test											
Reference	original layers, softmax + CE output	2×2 / 22	40.8k	1.5M	14	10	64.5	60.5	**60.9**	60.6	—
Width ×2	filters 32-64-128	2×2 / 22	118.7k	5.1M	32	9	68.2	63.8	64.9	63.9	+4.0
Width ×4	filters 64-128-256	2×2 / 22	412.6k	18.8M	73	9	72.0	66.0	66.3	65.6	+5.4
Depth −1	2 conv blocks (16-32)	6×6 / 10	18.2k	1.2M	11	10	50.4	49.1	50.0	48.3	−10.9
Depth +1	4 blocks (16-32-64-128), padding 1	2×2 / 46	122.9k	4.0M	22	10	70.8	65.0	66.2	65.3	+5.3
Two convs / block	two 3×3 convs per block, padding 1	4×4 / 36	89.3k	9.9M	38	10	47.8	48.0	48.4	46.8	−12.6
Padding	padding 1 ('same')	4×4 / 22	40.8k	2.8M	17	10	59.1	56.0	56.8	55.6	−4.1
Kernel 5×5	5×5 kernels, padding 2	4×4 / 36	82.5k	7.8M	23	10	65.8	63.8	64.7	64.3	+3.7

Model	Change vs. reference	Map / RF	Params	MACs	s/ep	Best ep	Train %	Val %	Test %	F1 %	Δ test
Average pooling	average- instead of max-pooling	2×2 / 22	40.8k	1.5M	15	10	53.9	51.3	52.6	50.8	−8.4
Strided conv	stride-2 convs instead of pooling	3×3 / 15	40.8k	0.5M	8	10	49.0	47.1	47.7	46.4	−13.2
BatchNorm	BatchNorm after each conv	2×2 / 22	41.0k	1.5M	20	8	67.8	65.9	66.3	66.0	+5.3
Dropout	dropout 0.3 before each FC layer	2×2 / 22	40.8k	1.5M	15	10	49.8	55.0	55.8	55.1	−5.1
Flatten head	flatten 64×2×2 instead of GAP	2×2 / 22	65.4k	1.5M	15	10	68.7	61.9	62.8	62.4	+1.9
Follow-up: BatchNorm added to the two deeper networks											
Depth +1 + BN	4 blocks, padding 1, BatchNorm	2×2 / 46	123.3k	4.0M	30	10	81.2	72.7	72.8	72.3	+11.9
Two convs / block + BN	two convs/block, padding 1, BN	4×4 / 36	89.8k	9.9M	44	10	78.5	74.2	**75.1**	74.8	+14.1
Combined model (chosen before any results were in)											
Combined	width ×2 + padding 1 + BatchNorm	4×4 / 22	119.1k	10.3M	53	9	71.6	70.0	70.5	69.7	+9.6

Table 1. All runs (seed 0, 10 epochs, best-validation-loss checkpoint). Train % is the running accuracy during that epoch; s/ep was timed separately on one idle CPU thread. Seed spread is about ±1 point (Table 2), so differences under ~2 points are not reliable.

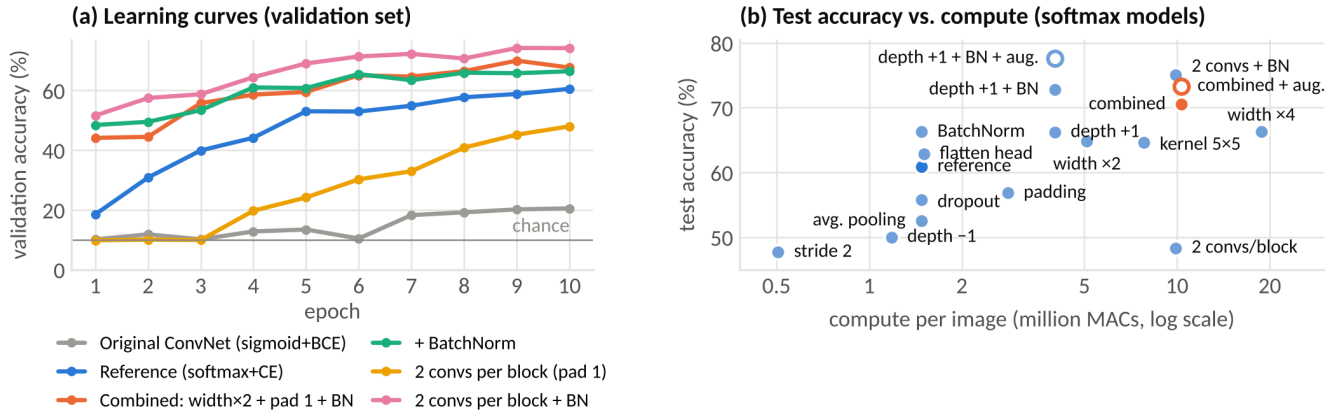


Figure 1. (a) Validation accuracy per epoch. The sigmoid model and the plain two-conv network both sit at chance for several epochs; with BatchNorm the same deep network ends highest. (b) Test accuracy against compute (log scale); upper left is more accuracy per MAC. Hollow markers: augmented (Section 5).

4. Discussion

Normalisation. Nothing else I tried came close to batch normalisation [2] for value: +5.3 points on seed 0, +5.8 ± 0.9 over three seeds ($p = 0.0012$), for 224 extra parameters and no extra MACs at all. Keeping each layer's activations well scaled is enough to let even this small learning rate move: one epoch in, the BatchNorm network was already at 48% validation accuracy against 19% for the reference. What I did not expect was how much it mattered to the deeper networks. The nine-layer version with two convolutions per block spent three epochs pinned at a loss of ln 10 and finished 12.6 points below the reference, which looked like proof it was too deep for 30,000 images. With BatchNorm the same layers reach 75.1%, my best run without augmentation, and the four-block network goes from 66.2% to 73.5 ± 1.8%. The depth was never the problem; training it was.

Width. Doubling every filter count (32-64-128) gave +4.0 points for 2.9× the parameters and 3.4× the MACs, which already looks like a poor trade. Doubling them again gave +5.4: only 1.4 points beyond width ×2, for 12.7× the reference MACs and about 5× its training time. That last gain is the same size as the seed-to-seed spread (0.4), so I would not read much into it. The wider model also began to over-fit: its validation loss rose after epoch 9 while training accuracy kept climbing, to 74.7% against 64.5% at epoch 10. Width clearly shows diminishing returns here.

Depth and receptive field. Depth behaved quite differently. Removing the third block cost 10.9 points, and the reason is visible in the geometry: the last units then see a 10×10 patch, and almost no CIFAR-10 object fits in a tenth of the frame. Adding a fourth block, which needs padding to survive the shrinking, pushes the receptive field to 46 px, wider than the image itself, and gained +5.3 — as much as width ×4 for a quarter of its MACs. I had assumed padding would help on its own for the same reason, and it did not: alone it cost 4.1 points. In hindsight that makes sense. Padding enlarges the final map to 4×4 without enlarging what any unit sees, so global average pooling ends up averaging four partial views of the object where it previously had one. Padding is worth having only when a further block spends the resolution it preserves. The 5×5 kernels tell the same story: at equal padding they beat 3×3 by 7.8 points, but at 2.8× the MACs, where the four-block 3×3 stack sees wider for half of that. Coverage of the object is what accuracy tracks, and small stacked kernels are the cheap way to buy it [3].

Downsampling. The lecture treats pooling and strided convolution as interchangeable ways of shrinking a map, so I tested both. Neither alternative survived contact with the data. Average pooling cost 8.4 points and was behind from the start (37.8% against 44.2% validation accuracy by epoch four). Max-pooling forwards the strongest response in each window, which is precisely the evidence that a feature is there, and sends the whole gradient back to it; averaging blunts both halves of that. Stride-2 convolutions were worse again, 13.2 points down, though they are genuinely cheap at 0.51M MACs, a third of the reference. The cost is that every later convolution runs on a smaller map and the final receptive field shrinks from 22 px to 15. In a network only three blocks deep that is not a trade worth making, which is consistent with all-convolutional networks needing far more depth before the same idea works [7].

Dropout [4] of 0.3 before each fully connected layer cost 5.1 points, which is the clearest sign in the whole table that I was treating the wrong illness. Dropout answers over-fitting, and this model was under-fitting: 49.8% on training against 55.0% on validation. All the extra noise did was slow learning further. Swapping global average pooling [5] for flattening went the other way and gained 1.9 points, because on a map as small as 2×2 the averaging throws away *where* a feature fired; keeping that costs 24.6k parameters in the first FC layer.

I had picked a combined model before seeing any results — width ×2, padding and BatchNorm together — assuming the gains would add up. They did not. It managed 70.5% on seed 0, but over three seeds it is no better than BatchNorm alone (68.9 ± 1.6% against 67.1 ± 0.8%) at 7× the compute, and two of its three ingredients turned out to be weak on their own. Depth is the better answer: two convolutions per block reach 75.1% at 9.9M MACs, and the four-block network beats the combined model by 4.6 points over three seeds ($p = 0.031$) using 39% of its MACs.

One caveat colours the whole table: these models are mostly still under-fitting. Training and validation accuracy sit within ±7 points of each other for every model but one, and 16 of the 17 softmax runs were still improving at epoch 9 or 10 when training stopped. That explains the pattern above: whatever adds capacity or eases optimisation helps, whatever restrains the model hurts. Only two runs push back, both the ones with the most capacity for 30,000 images — the four-block BatchNorm network, with the widest train–validation gap in the table (81.2% against 72.7%), and width ×4, whose validation loss turns up after epoch 9. Section 5 tests whether augmentation helps them most.

Breaking the best network down by class (Fig. 2) shows it beating the reference in 9 of the ten, with the largest gains on automobile (+29 points), deer (+22) and horse (+21). The mistakes it still makes are the ones a person would make on a thumbnail: cat and dog go both ways (5% and 26%), automobile is read as truck 10% of the time and deer as horse 9%. Dog (43%) and bird (57%) stay hardest, the two categories whose members vary most in shape and colour. Averages hide a lot, though: the combined model improved nine classes yet lost 11 points on cat, predicting dog 612 times and frog 1,178, against 1,000 of each.

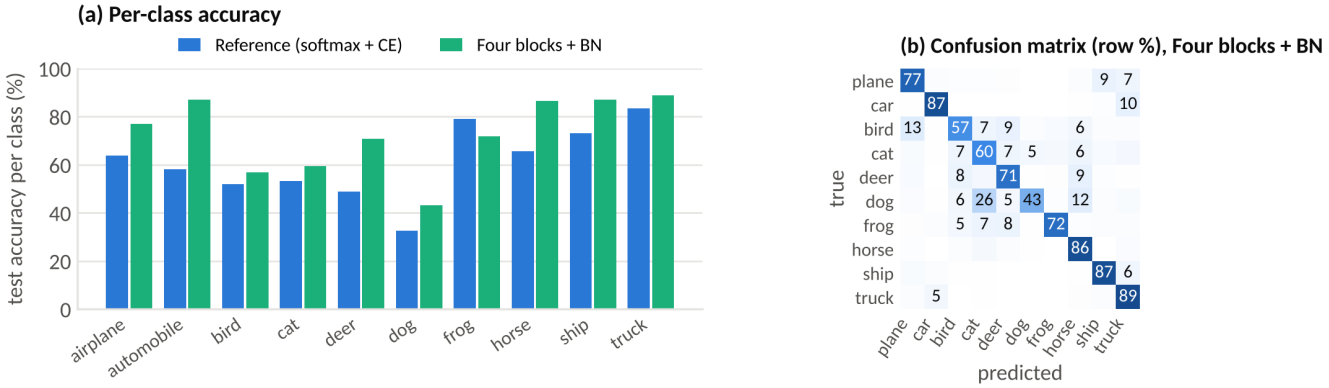


Figure 2. (a) Per-class test accuracy, reference against the four-block BatchNorm network (seed 0). (b) Its confusion matrix, % of each true class (cells under 5% blank).

5. Robustness check and training improvements

One run per model is thin evidence for a difference of a point or two, so I re-ran the four models the argument rests on with seeds 1 and 2 (Table 2). The seed-to-seed standard deviation is 0.4–1.8 points, enough to settle two questions. The BatchNorm gain is far larger than that noise (Welch's t-test, $p = 0.0012$) and survives; the combined model's +4.2-point lead over BatchNorm on seed 0 does not, shrinking to +1.9 on average at $p = 0.18$. Had I stopped at one seed I would have reported it as a finding. I then tested the under-fitting diagnosis directly by retraining the combined and four-block models with the standard remedies: random 32×32 crops from images padded by 4 pixels, horizontal flips, a cosine schedule [6] and 15 epochs. They reached 73.3% and **77.6%** (macro-F1 77.6), with training accuracy still close to validation accuracy, so even these are not converged.

Model (10 epochs)	Seed 0	Seed 1	Seed 2	Mean ± std	Δ mean vs. reference
Reference (softmax)	60.9	61.7	61.0	61.2 ± 0.4	—
BatchNorm	**66.3**	**67.1**	**67.8**	**67.1 ± 0.8**	**+5.8**
Combined (width ×2 + pad + BN)	70.5	67.2	69.0	68.9 ± 1.6	+7.7
Depth +1 + BN (follow-up)	**72.8**	**75.6**	**72.2**	**73.5 ± 1.8**	**+12.3**
Notebook as given: sigmoid, red only, 1 epoch (5 seeds)	10.3 / 9.9 / 11.3			10.4 ± 0.6	(chance level = 10)
Combined + augmentation + cosine LR, 15 epochs	73.3				+12.1
Depth +1 + BN + augmentation + cosine LR, 15 epochs	77.6				+16.4

Table 2. Test accuracy (%) over seeds (best-validation-loss checkpoint). The last two rows also change the training procedure, so their Δ includes that change.

6. Conclusion and limitations

The notebook cannot learn in one epoch for two reasons that have nothing to do with its architecture: the sigmoid and BCE output, and a loader that shows it only the red channel. Fix those and the design choices begin to matter, though not as "bigger is better" would predict. The cheapest change was the most useful and the expensive ones mostly were not: BatchNorm costs 224 parameters and no extra arithmetic, while width ×4 bought 1.4 points over width ×2 for 13× the compute. With normalisation in place depth beat filters — 73.5% at 4.0M MACs against 68.9% at 10.3M — and receptive field, not parameter count, seems to be why. Training choices counted for as much as layers: augmentation and a cosine schedule lifted that same network to 77.6%, above the 76.2% of the course's pretrained EfficientNet checkpoint on the same test set. Given more time I would seed every variant, train past the point where validation loss flattens, and fix *EffNet_clf*, which applies the ImageNet normalisation to 0–255 pixels.

Limitations. Most variants were trained once, for 10 epochs on a CPU, so these are early-training numbers rather than converged ones, and the split uses 30,000 images where CIFAR-10 is normally quoted on 45,000 — compare them with each other, not with the literature.

References. [1] A. Krizhevsky, *Learning multiple layers of features from tiny images*, Univ. of Toronto, 2009. [2] S. Ioffe, C. Szegedy, "Batch normalization," ICML 2015. [3] K. Simonyan, A. Zisserman, "Very deep convolutional networks for large-scale image recognition," ICLR 2015. [4] N. Srivastava et al., "Dropout," JMLR 15, 2014. [5] M. Lin, Q. Chen, S. Yan, "Network in network," ICLR 2014. [6] I. Loshchilov, F. Hutter, "SGDR," ICLR 2017. [7] J. T. Springenberg et al., "Striving for simplicity: the all convolutional net," ICLR Workshop 2015.